

1.0 Introduction

Predictive maintenance is an advanced maintenance strategy that focuses on forecasting equipment failures before they occur, allowing maintenance activities to be done only when necessary. Unlike reactive maintenance, which repairs machines after breakdowns occur, and preventive maintenance, which follows scheduled servicing intervals, predictive maintenance continuously monitors equipment conditions using real-time sensor data and analytical techniques to identify early signs of failure (V. Noor Mohammed, Abdulateef and Hamad 2023; IBM 2023). By analysing operational parameters such as temperature, vibration, pressure, sound, and rotational speed, predictive maintenance enables industries to minimize unexpected downtime, improve reliability, extend equipment lifespan, and reduce unnecessary maintenance costs (Cybertrol Engineering 2025; Siemens 2026; IBM 2023).

In industrial machines and manufacturing settings, predictive maintenance plays an important role in ensuring continuous and efficient production operations. Improper maintenance of industrial machines may lead to overheating, abnormal vibrations, and mechanical wear, which can interrupt manufacturing processes and increase operational costs (Siemens 2026). Therefore, real-time monitoring systems are required to detect abnormalities at an early stage and provide immediate feedback to operators. Sensor nodes deployed within industrial machines can collect telemetry data such as temperature, vibration, pressure and RPM values, which are then processed to evaluate equipment health and predict potential failures before they escalate into critical issues (u-blox 2023; Cybertrol Engineering 2025).

To support predictive maintenance in IIoT environment, this project adopts MQTT (Message Queuing Telemetry Transport) as the communication protocol between devices and monitoring systems. MQTT is a lightweight publish-subscribe messaging protocol designed for low-bandwidth, high-latency, and unreliable networks, making it highly suitable for IoT applications involving resource-constrained devices (Potey 2023). Furthermore, MQTT provides efficient bi-directional communication between devices and cloud systems while supporting scalability, reliable message delivery, real-time communication, and low network overhead (MQTT 2024; Potey 2023). The protocol also supports Quality of Service (QoS) levels, persistent sessions, encryption using TLS, authentication mechanisms, and access control features, which are important for secure industrial IoT deployments (MQTT 2024; Potey 2023).

This project develops an MQTT-based predictive maintenance system for industrial machines using Python clients and a graphical dashboard interface. The system simulates real-time telemetry monitoring by generating machine temperature and vibration data and transmitting the information through private and public MQTT topics. A predictive maintenance monitoring client – User 2 analyses the telemetry data and automatically generates maintenance commands based on predefined thresholds. In addition, a GUI dashboard is implemented to visualize telemetry data, maintenance alerts, and machine conditions in real time, enabling operators to monitor the industrial machine more effectively.

1.1 Problem Statement

Industrial machines are widely used in modern manufacturing environments to improve productivity, precision, operational efficiency and automation capabilities. However, continuous machine operation may result in mechanical wear, overheating, excessive vibration, and component degradation over time. If these issues are not detected early, they may lead to unexpected equipment failures, unplanned production downtime, increased maintenance costs, reduced operational efficiency, and potential safety risks within the manufacturing environment.

Traditional maintenance approaches such as reactive maintenance only address problems after failures occur, often resulting in costly repairs and production interruptions. Preventive maintenance, while more proactive, relies on fixed maintenance schedules that may result in unnecessary servicing or missed early warning signs of equipment deterioration.

In industrial environments, manually monitoring machine conditions can also be inefficient and unreliable, especially when multiple machines are operating simultaneously. Maintenance personnel may not be able to continuously observe critical operating parameters such as temperature and vibration in real time. As a result, abnormal operating conditions may remain undetected until serious mechanical damage occurs.

Furthermore, modern industrial systems increasingly rely on Internet-connected IoT communication technologies such as MQTT to enable remote monitoring and automation. While MQTT provides lightweight and efficient communication between IoT devices, internet-facing MQTT infrastructures may also introduce cybersecurity risks including unauthorised access, message tampering, denial-of-service attacks, data interception, and system disruption if appropriate security controls are not implemented.

Therefore, there is a need for a predictive maintenance monitoring system capable of:

- continuously monitoring industrial machine telemetry data,
- detecting abnormal operating conditions in real time,
- automatically generating maintenance responses,
- and securely transmitting telemetry data through MQTT-based communication architecture.

1.2 Objectives

The objective of this project is to design and implement an MQTT-based predictive maintenance system for industrial machines that enables real-time telemetry monitoring, condition analysis, and automated maintenance decision-making. By utilising MQTT communication between multiple clients, the system demonstrates how temperature and vibration data can be analysed to identify potential machine faults, reduce unexpected downtime, and improve maintenance effectiveness. In addition, an interactive dashboard is developed to provide operators with real-time machine visualisation, maintenance alerts, and operational control capabilities.

2.0 System Architecture

The MQTT-based Predictive Maintenance System is designed using a publish-subscribe architecture to enable real-time communication between multiple MQTT clients through a central MQTT broker. The system consists of three main components: the Machine Telemetry Publisher (User 1), the Predictive Maintenance Monitoring Client (User 2), and the GUI Dashboard Client. All components communicate through a Mosquitto MQTT broker hosted at IP address 192.168.12.100 using the MQTT protocol.

The Machine Telemetry Publisher is responsible for generating simulated machine telemetry data, including temperature and vibration readings, and publishing the information to both private and public MQTT topics. The Predictive Maintenance Monitoring Client subscribes to the telemetry data, analyses machine operating conditions using predefined thresholds, and generates predictive maintenance decisions and operational commands. The GUI Dashboard Client provides a graphical interface that allows operators to monitor machine conditions in real time, view maintenance alerts, and perform control actions such as emergency stop and maintenance reset.

Communication between system components is achieved using MQTT topics that separate telemetry data, predictive maintenance analysis, maintenance commands, acknowledgements, reset operations, and system status updates. This architecture enables loose coupling between clients, allowing each component to operate independently while maintaining real-time information exchange through the MQTT broker.

Figure 2.1 illustrates the overall architecture of the MQTT-based Predictive Maintenance System and the communication flow between all MQTT clients.

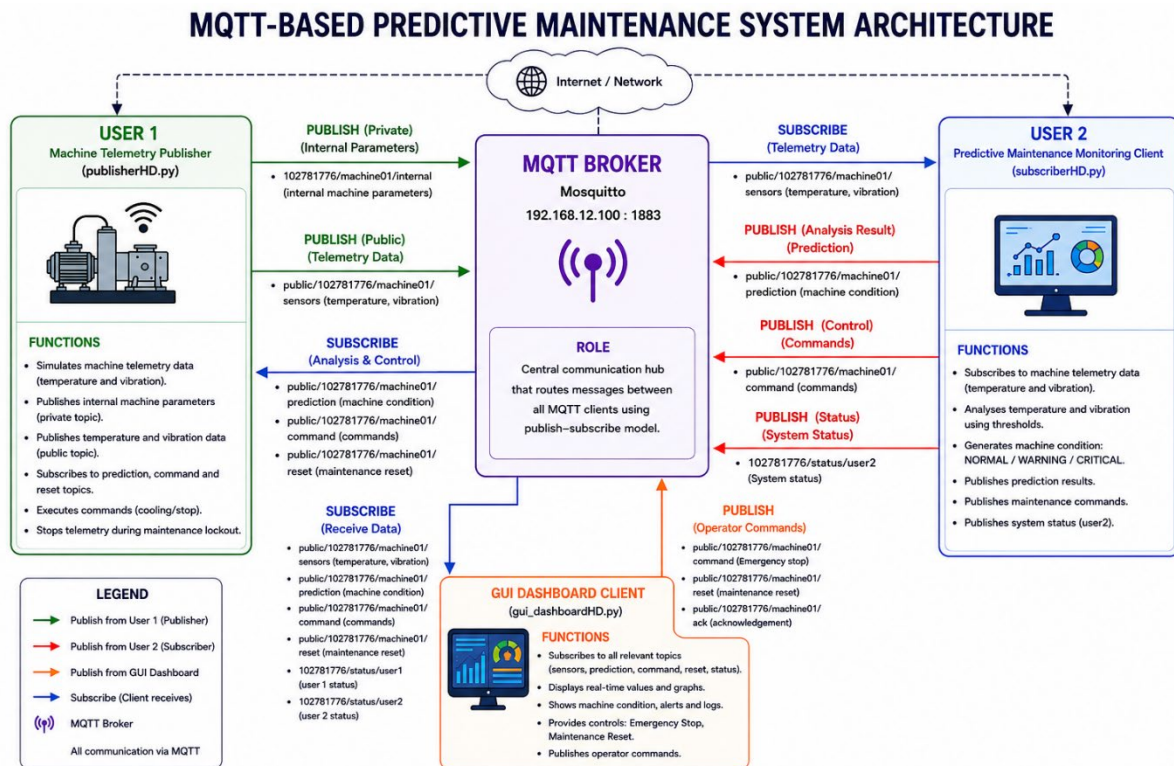


Figure 1: System architecture of the MQTT-based Predictive Maintenance System

2.1 MQTT Broker Used In This Project

The project uses the Mosquitto MQTT broker to manage communication between all MQTT clients within the predictive maintenance system.

The MQTT broker acts as the central message broker responsible for:

- receiving published MQTT messages,
- managing MQTT topics,
- and distributing messages to subscribed MQTT clients using publish-subscribe communication architecture.

For this project, the MQTT broker was hosted using the following local network IP address:

192.168.12.100

The broker operates on the default MQTT port:

1883

The MQTT broker enables communication between:

- User 1 publisher client (publisherHD.py)
- User 2 predictive maintenance subscriber (subscriberHD.py)
- GUI monitoring dashboard (gui.py)

The broker allows MQTT clients to publish and subscribe to both private and public MQTT topics for telemetry monitoring, predictive maintenance analysis, and command handling.

2.2 MQTT Topics and Messages

This section explains the MQTT topic structure used for communication between the publisher, subscriber and GUI dashboard. The project uses both private and public MQTT topics to separate confidential machine telemetry data from publicly shared predictive maintenance data.

Private MQTT Topic

The following private MQTT topic is used to transmit confidential internal machine telemetry data:

102781776/machine01/internal

This topic contains:

- motor RPM
- oil level percentage
- runtime hours
- internal bearing temperature
- vibration readings

The topic is restricted to the user's own topic namespace and represents sensitive operational telemetry data.

Public MQTT Topic 1 – Sensor Data

The following public MQTT topic is used to transmit public machine sensor telemetry data:

`public/102781776/machine01/sensors`

This topic contains:

- temperature readings
- vibration readings
- machine running state

This topic allows other MQTT clients to subscribe and analyse machine operating conditions.

Public MQTT Topic 2 – Predictive Maintenance Results

The following public MQTT topic is used to transmit predictive maintenance analysis results:

`public/102781776/machine01/prediction`

This topic contains:

- NORMAL state
- WARNING state
- CRITICAL state
- Maintenance-required notifications
- Predictive maintenance responses

Command Topic

The following MQTT topic is used for command-and-control communication:

`public/102781776/machine01/command`

This topic contains commands such as:

- ENABLE_COOLING_AND_SCHEDULE_MAINTENANCE
- STOP_MACHINE_AND_INSPECT
- MAINTENANCE_REQUIRED

Reset Topic

The following MQTT topic is used to reset the maintenance lockout state:

`public/102781776/machine01/reset`

Acknowledgement Topic

The following MQTT topic is used for technician acknowledgement messages:

`public/102781776/machine01/ack`

Status Topics

The following topics are used to monitor MQTT client status:

`102781776/status/user1`

`102781776/status/user2`

These topics display:

- online status
- offline status
- machine running state
- maintenance lockout state

2.3 User 1 – Machine Telemetry Publisher (publisherHD.py)

User 1 acts as the primary data source, which is also a simulated industrial machine integrated with multiple sensor nodes, specifically temperature and vibration, within the predictive maintenance system. As discussed in Section 1.1, one of the key challenges in industrial environments is the difficulty of continuously monitoring machine operating conditions in real time. Without continuous monitoring, abnormal conditions such as excessive temperature and vibration may remain undetected until severe mechanical degradation or equipment failure occurs. Therefore, User 1 was developed to simulate machine operation and provide a continuous stream of telemetry data for predictive maintenance analysis.

```
# OBJ 3: Subscribe to Topic 2 in the public channel and display generated messages
# User 1 also subscribes to the command topic to receive any commands sent by User 2 based on the predictions.
client.subscribe(public_topic2)
client.subscribe(command_topic)
client.subscribe(reset_topic)
```

Figure 2: Subscribed topics for publisherHD.py

The publisher establishes a connection to the MQTT broker hosted at IP address 192.168.12.100 using the default MQTT communication port 1883. Once connected, the client automatically generates machine telemetry values including bearing temperature, vibration readings, motor RPM, oil level percentage, runtime hours, machine running status, and maintenance status. These values are dynamically generated using Python-based simulation logic to represent realistic industrial machine behaviour.

Furthermore, gradual fluctuations are incorporated into the simulation to prevent unrealistic sudden changes in sensor readings and to better reflect actual machine operating conditions.

```
temperature += random.uniform(-1.5, 1.5)
vibration += random.uniform(-0.3, 0.3)
```

Figure 3: Gradual fluctuations set to both sensor readings

To support structured communication, the publisher transmits telemetry data through both private and public MQTT topics. The private MQTT topic, namely 102781776/machine01/internal is used to publish internal machine telemetry data.

```

# OBJ 1: User 1 automatically generate and post messages to a topic in the private channel.
# creates simulated internal machine diagnostics data
# The payload produced here is later published to: 102781776/machine01/internal
private_payload = {
    "student_id": student_id,
    "machine_id": machine_id,
    "motor_rpm": random.randint(1300, 1600),
    "bearing_temperature_c": temperature,
    "vibration_x_mm_s": vibration,
    "oil_level_percent": random.randint(40, 100),
    "runtime_hours": random.randint(800, 2500),
    "machine_running": machine_running,
    "maintenance_required": maintenance_required
}

```

Figure 4: Private payload to be published to private_topic

Topic: 102781776/machine01/internal QoS: 0

```

{
    "student_id": "102781776",
    "machine_id": "machine01",
    "motor_rpm": 1306,
    "bearing_temperature_c": 59.41,
    "vibration_x_mm_s": 2.21,
    "oil_level_percent": 93,
    "runtime_hours": 968,
    "machine_running": true,
    "maintenance_required": false
}

```

Figure 5: Expected json output when User 1 publishes private data to private_topic

This topic contains sensitive operational information such as motor RPM, oil level percentage, runtime hours, machine status, and maintenance-related information. By separating this information from publicly accessible data, the system demonstrates how MQTT topic structures can be used to segregate confidential operational data from monitoring data intended for wider system access.

In addition to the private topic, User 1 publishes machine sensor telemetry data to public/102781776/machine01/sensors, which serves as the primary telemetry source for machine condition monitoring and predictive maintenance analysis. The published payload contains temperature readings, vibration readings, and machine running state information. These telemetry values are transmitted continuously at fixed intervals, thereby simulating real-time machine monitoring within an industrial environment.

```

User 1 published public sensor data to:
public/102781776/machine01/sensors
{
    "student_id": "102781776",
    "machine_id": "machine01",
    "temperature_c": 66.14,
    "vibration_mm_s": 2.39,
    "machine_running": true
}

```

Figure 6: Expected json output when User 1 publishes sensor data to public_topic1

Although the publisher generates multiple telemetry parameters, only temperature and vibration readings are published to the public telemetry topic. This design decision was made because temperature and vibration are the primary indicators used for predictive maintenance analysis within the system. Parameters such as motor RPM, oil level percentage, and runtime hours remain within the private MQTT topic as they represent internal operational information that is not required by other clients for maintenance decision-making. Consequently, the system reduces unnecessary data exposure while maintaining the separation between public monitoring data and sensitive machine telemetry.

As a result, other MQTT clients can subscribe to the telemetry stream and perform further analysis on machine operating conditions.

```
# OBJ 1: User 1 automatically generate and post messages to a topic in the private channel.
# creates simulated internal machine diagnostics data
# The payload produced here is later published to: 102781776/machine01/internal

private_payload = {
    "student_id": student_id,
    "machine_id": machine_id,
    "motor_rpm": random.randint(1300, 1600),
    "bearing_temperature_c": temperature,
    "vibration_x_mm_s": vibration,
    "oil_level_percent": random.randint(40, 100),
    "runtime_hours": random.randint(800, 2500),
    "machine_running": machine_running,
    "maintenance_required": maintenance_required
}

# OBJ 2: User 1 automatically generate and post messages to <Topic 1> in a public channel.
# extracts the public temperature and vibration data from the private machine payload.
# This public payload is later published to: public/102781776/machine01/sensors

public_payload = {
    "student_id": student_id,
    "machine_id": machine_id,
    "temperature_c": temperature,
    "vibration_mm_s": vibration,
    "machine_running": machine_running
}

client.publish(
    private_topic,
    json.dumps(private_payload, indent=4)
)

client.publish(
    public_topic1,
    json.dumps(public_payload, indent=4)
)
```

Figure 7: Payload sending depending on the topics published and subscribed

Besides publishing telemetry data, User 1 also subscribes to predictive maintenance and control topics. The following topic is used to receive predictive maintenance analysis generated by the monitoring client: public/102781776/machine01/prediction. Messages received through this topic provide machine condition assessments such as NORMAL, WARNING, CRITICAL, and MAINTENANCE_REQUIRED. Consequently, the publisher can display and respond to predictive maintenance decisions generated from the machine telemetry data.


```

User 1 received message
Topic: public/102781776/machine01/prediction
Payload:
{
  "machine_id": "machine01",
  "temperature_c": 57.78,
  "vibration_mm_s": 2.75,
  "condition": "NORMAL",
  "risk_score": 15,
  "recommended_action": "CONTINUE_OPERATION",
  "reason": "Machine is operating normally"
}

```

Figure 8: Expected json output when User 1 subscribes public_topic2

Furthermore, User 1 subscribes to public/102781776/machine01/command, which is responsible for transmitting operational commands such as automatic cooling activation, emergency stop requests, and maintenance lockout instructions. When a critical maintenance condition or emergency stop command is received, the publisher automatically suspends telemetry generation and places the machine into a maintenance lockout state. This behaviour simulates how industrial control systems may prevent continued operation when unsafe operating conditions are detected.

```

# User 1 processes messages based on the topic they were received on and send payload responses accordingly
if message.topic == command_topic:
    command = payload.get("command", "")
    # processes stop command to stop machine state and publish readings
    if command == "STOP_MACHINE_AND_INSPECT":
        machine_running = False
        print("\nEMERGENCY STOP command received.")
        print("Machine has stopped publishing sensor readings.")

        client.publish(
            status_topic,
            json.dumps(
                {
                    "user": "User 1",
                    "status": "stopped",
                    "machine_running": machine_running
                },
                indent=4
            ),
            retain=True
        )

```

Figure 9: Command action triggered during STOP_MACHINE_AND_INSPECT mode is activated

```

User 1 received message
Topic: public/102781776/machine01/command
Payload:
{
  "machine_id": "machine01",
  "command": "STOP_MACHINE_AND_INSPECT",
  "issued_by": "gui_dashboardHD",
  "reason": "Emergency stop button pressed by technician1"
}

EMERGENCY STOP command received.
Machine has stopped publishing sensor readings.

```

Figure 10: Expected json output when User 1 subscribes command_topic

```

Topic: 102781776/status/user1  QoS: 0  Retained
{
  "user": "User 1",
  "status": "stopped",
  "machine_running": false
}

```

Figure 11: Expected json output when User 1 publishes status_topic when emergency stop / crucial threshold is ON

The publisher also subscribes to public/102781776/machine01/reset, which allows authorised personnel to restore normal operation after maintenance activities have been completed. Upon receiving a valid reset command by pressing the blue button under quick actions section, telemetry generation resumes and the machine returns to its normal operating state.

```

# processes reset command to update machine state and publish status updates with json
elif message.topic == reset_topic:
    reset_state = payload.get("reset_maintenance_state", False)

    if reset_state is True:
        machine_running = True
        maintenance_required = False

        print("\nRESET command received.")
        print("Machine operation resumed.")

        client.publish(
            status_topic,
            json.dumps(
                {
                    "user": "User 1",
                    "status": "running_after_reset",
                    "machine_running": machine_running
                },
                indent=4
            ),
            retain=True
        )

```

Figure 12: Machine operation reset back to normal upon receiving a valid reset command

```

User 1 received message
Topic: public/102781776/machine01/reset
Payload:
{
  "machine_id": "machine01",
  "reset_maintenance_state": true,
  "operator": "technician1"
}

RESET command received.
Machine operation resumed.

```

Figure 13: Button pressed acknowledgement output when the user presses "RESET MAINTENANCE STATE"

```

Topic: 102781776/status/user1   QoS: 0

{
  "user": "User 1",
  "status": "running_after_reset",
  "machine_running": true
}

```

Figure 14: Expected json output when User 1 publishes status_topic when the reset button is pressed

In addition, User 1 publishes connection and machine status information through 102781776/status/user1. This topic enables other MQTT clients to monitor machine availability, operational status, and maintenance conditions.

```

client.publish(
    status_topic,
    json.dumps(
        {
            "user": "User 1",
            "status": "online",
            "machine_running": machine_running
        },
        indent=4
    ),
    retain=True
)

```

Figure 15: Status topic to indicate machine availability

```

Topic: 102781776/status/user1   QoS: 0

{
  "user": "User 1",
  "status": "online",
  "machine_running": true
}

```

Figure 16: Expected json output when User 1 publishes status_topic when the User 1 is back on

To improve reliability, MQTT Last Will and Testament (LWT) functionality is also implemented. If the publisher disconnects unexpectedly from the MQTT broker, an automatic status message is transmitted to notify other connected clients of the communication failure.

```

# will_set is used to define a last will message that will be published if the client disconnects unexpectedly
client.will_set(
    status_topic,
    json.dumps(
        {
            "user": "User 1",
            "status": "offline unexpectedly",
            "machine_running": False
        },
        indent=4
    ),
    retain=True
)

```

Figure 17: LWT functionality

The telemetry produced by the publisher provides the real-time operational data required for machine condition monitoring, predictive maintenance analysis, automated maintenance responses, and dashboard visualisation.

2.4 User 2 – Predictive Maintenance Monitoring Client (subscriberHD.py)

User 2 serves as the analytical component of the predictive maintenance system. While User 1 is responsible for generating and publishing machine telemetry data, User 2 continuously receives the telemetry data and evaluates the machine's operating condition based on predefined maintenance thresholds.

```
def on_connect(client, userdata, flags, rc):
    if rc == 0:
        print("User 2 connected to MQTT broker")
        # OBJ 2: User 2 subscribes to Topic 1 to receive and display sensor readings generated by User 1.
        # Topic 1: public/102781776/machine01/sensors
        client.subscribe(public_topic1)
        print("Subscribed to:", public_topic1)

        client.publish(
            status_topic,
            json.dumps({"user": "User 2", "status": "online"}, indent=4),
            retain=True
        )
    else:
        print("Connection failed with code:", rc)
```

Figure 18: Subscribed topic for subscriberHD.py

The client establishes a connection to the MQTT broker hosted at **192.168.12.100** using MQTT port **1883**. Once connected, it subscribes to `public/102781776/machine01/sensors`. This topic contains temperature readings, vibration readings, and machine running state generated by User 1, as shown in Figure 5.

The telemetry data published by User 1 provides the information required for predictive maintenance analysis. Temperature and vibration were selected because they are widely used condition-monitoring indicators in industrial environments. Excessive temperature may indicate overheating, poor lubrication, or excessive mechanical load, while abnormal vibration may indicate bearing wear, imbalance, shaft misalignment, or other mechanical faults. By continuously monitoring these parameters, the system can assess machine health and detect abnormal operating conditions at an early stage.

Upon receiving telemetry data, User 2 compares the readings against predefined maintenance thresholds. Based on the analysis results, the machine condition is classified into one of three operating states:

- **NORMAL** – Machine operating conditions remain within acceptable limits.
- **WARNING** – Machine conditions are approaching abnormal levels and maintenance planning should be considered.
- **CRITICAL** – Machine conditions have exceeded safe operating thresholds and immediate action may be required.

```

# Prediction logic to analyze machine state based on temperature and vibration readings
# OBJ 1: Automatically generate prediction messages based on Topic 1 and publish to Topic 2
# This function analyses incoming temperature and vibration readings received from Topic 1: public/102781776/machine01/sensors
# It generates a prediction result: NORMAL, WARNING or CRITICAL.
def analyse_machine(temperature, vibration):
    # critical condition if temperature is 85 or above, or vibration is 6 or above
    if temperature >= 85 or vibration >= 6:
        return {
            "condition": "CRITICAL",
            "risk_score": 90,
            "recommended_action": "STOP_MACHINE_AND_INSPECT",
            "reason": "Temperature or vibration is too high"
        }
    # warning condition if temperature is 75 or above, or vibration is 4.5 or above
    elif temperature >= 75 or vibration >= 4.5:
        return {
            "condition": "WARNING",
            "risk_score": 60,
            "recommended_action": "ENABLE_COOLING_AND_SCHEDULE_MAINTENANCE",
            "reason": "Temperature or vibration is above normal range"
        }
    # normal condition if temperature is below 75 and vibration is below 4.5
    else:
        return {
            "condition": "NORMAL",
            "risk_score": 15,
            "recommended_action": "CONTINUE_OPERATION",
            "reason": "Machine is operating normally"
        }

```

Figure 19: Machine conditions based on current operating states

After completing the analysis, User 2 publishes the prediction results to public/102781776/machine01/prediction. The published prediction messages include the machine condition, risk score, recommended action, and the reason for the generated decision. These messages are subsequently received by User 1 and the GUI Dashboard Client.

```

# OBJ 1.1: analyses the machine condition based on the received sensor data (Topic 1) and
# generates a prediction payload
analysis = analyse_machine(temperature, vibration)

prediction_payload = {
    "machine_id": machine_id,
    "temperature_c": temperature,
    "vibration_mm_s": vibration,
    "condition": analysis["condition"],
    "risk_score": analysis["risk_score"],
    "recommended_action": analysis["recommended_action"],
    "reason": analysis["reason"]
}

# publishes the prediction result to Topic 2: public/102781776/machine01/prediction for User 1 to receive and act upon
client.publish(
    public_topic2,
    json.dumps(prediction_payload, indent=4)
)

```

Figure 20: Predictive analysis machine payload

Topic: public/102781776/machine01/prediction QoS: 0

```
{
  "machine_id": "machine01",
  "temperature_c": 67.06,
  "vibration_mm_s": 3.88,
  "condition": "NORMAL",
  "risk_score": 15,
  "recommended_action": "CONTINUE_OPERATION",
  "reason": "Machine is operating normally"
}
```

Figure 21: Expected json output when User 2 publishes public_topic2 based on sensor readings based on public_topic1

In addition to generating condition assessments, User 2 also publishes maintenance and operational commands through public/102781776/machine01/command.

The generated commands include:

- ENABLE_COOLING_AND_SCHEDULE_MAINTENANCE
- STOP_MACHINE_AND_INSPECT
- MAINTENANCE_REQUIRED

These commands allow the system to automatically recommend corrective actions when abnormal operating conditions are detected. For example, when warning temperature thresholds are exceeded, the system recommends cooling and maintenance scheduling. Similarly, when critical operating conditions are detected, emergency stop or maintenance-required commands may be generated to prevent further machine deterioration.

```
# extra functionality: auto command generation
# If the prediction is WARNING or CRITICAL, User 2 publishes a command to the command topic.
# complements the system by adding automatic cooling or emergency response behaviour.
if analysis["condition"] == "WARNING" or analysis["condition"] == "CRITICAL":
    # constructs command payload with machine ID, recommended action, and reason for the command based on the analysis
    command_payload = {
        "machine_id": machine_id,
        "command": analysis["recommended_action"],
        "reason": analysis["reason"]
    }

    client.publish(
        command_topic,
        json.dumps(command_payload, indent=4)
    )

    print("\nUser 2 published command to:")
    print(command_topic)
    print(json.dumps(command_payload, indent=4))
```

Figure 22: Recommended action prompted based on current operating conditions

```

Topic: public/102781776/machine01/command   QoS: 0

{
  "machine_id": "machine01",
  "command": "ENABLE_COOLING_AND_SCHEDULE_MAINTENANCE",
  "reason": "Temperature or vibration is above normal
range"
}

```

Figure 23: Expected json output when User 2 publishes command_topic

Furthermore, User 2 publishes its connection status to 102781776/status/user2. This status topic enables other MQTT clients to monitor the availability of the predictive maintenance monitoring service.

```

client.publish(
    status_topic,
    json.dumps({"user": "User 2", "status": "online"}, indent=4),
    retain=True
)

```

Figure 24: Status topic to indicate User 2's availability

```

Topic: 102781776/status/user2   QoS: 0   Retained

{
  "user": "User 2",
  "status": "online"
}

```

Figure 25: Expected json output for User 2's status

MQTT Last Will and Testament (LWT) functionality is also implemented similarly to Figure 8 to notify connected clients if the monitoring client disconnects unexpectedly from the broker.

By continuously analysing telemetry data and generating automated maintenance responses, the client enables early fault detection, proactive maintenance planning, and reduced machine downtime.

2.5 GUI Dashboard Client (gui.py)

The GUI Dashboard Client serves as the Human Machine Interface (HMI) of the predictive maintenance system and provides a centralised platform for real-time monitoring, predictive maintenance visualisation, and operator interaction. The use case for Python libraries is illustrated with the code comments as shown below.

```
# python code (extra functionality) for gui_dashboardHD.py
from http import client # for handling HTTP client functionality if needed for future extensions such as REST API integration
import json # for handling JSON data in MQTT messages
from collections import deque # for storing recent sensor data for graphing
from datetime import datetime # for timestamping log messages and graph data points
import tkinter as tk # for creating the GUI dashboard
from tkinter import messagebox # for popup alarms

import paho.mqtt.client as mqtt # for MQTT communication between publisher and subscriber

from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg # for embedding Matplotlib graphs in Tkinter
from matplotlib.figure import Figure # for creating Matplotlib figures for live graphing
```

Figure 26: Libraries used for this dashboard client with the use cases

Unlike User 1 and User 2, which primarily perform telemetry generation and condition analysis, the GUI Dashboard Client focuses on presenting information in an intuitive format that allows maintenance personnel to monitor machine conditions and respond to abnormal operating situations.

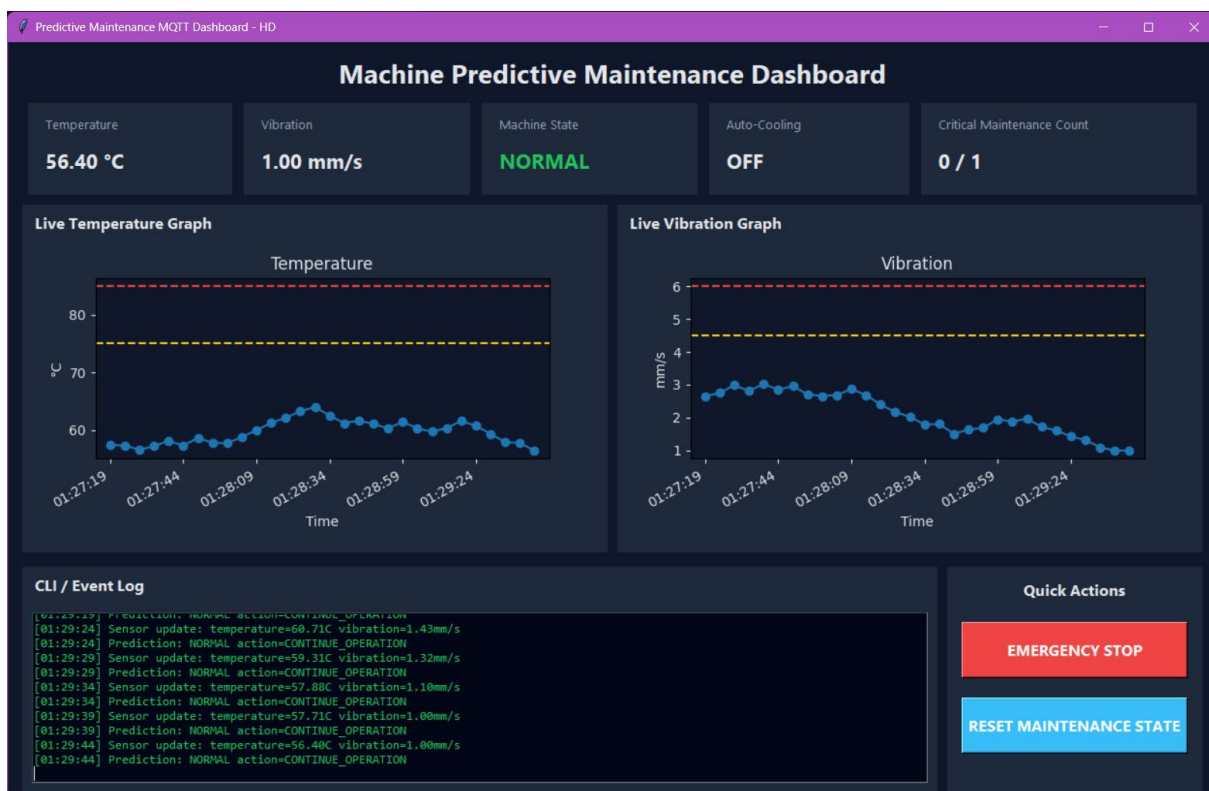


Figure 27: Running GUI Dashboard Client

The dashboard is developed using Python Tkinter and communicates with the MQTT broker hosted at 192.168.12.100 using MQTT publish-subscribe communication. By operating as an independent MQTT client, the dashboard can both subscribe to and publish MQTT messages, allowing it to actively participate in machine monitoring and maintenance operations.

The dashboard subscribes to the following MQTT topics, similarly, as shown in Figure 14:

- public/102781776/machine01/sensors
- public/102781776/machine01/prediction
- public/102781776/machine01/command
- 102781776/status/user1
- 102781776/status/user2


```

[01:34:46] Subscribed to public/102781776/machine01/sensors
[01:34:46] Subscribed to public/102781776/machine01/prediction
[01:34:46] Subscribed to public/102781776/machine01/command
[01:34:46] Subscribed to 102781776/status/user1
[01:34:46] Subscribed to 102781776/status/user2
[01:34:46] Status update: {'user': 'User 1', 'status': 'stopped', 'machine_running': False}
[01:34:46] Status update: {'user': 'User 2', 'status': 'online'}

```

Figure 28: Subscribed MQTT topics and Users' status shown on the CLI / Event Log

These subscriptions enable the dashboard to receive live telemetry data, predictive maintenance decisions, operational commands, and client status information. Consequently, machine activity can be monitored in real time through a single interface.

The dashboard displays several machine monitoring indicators including:

- Temperature readings
- Vibration readings
- Machine operating state
- Auto-cooling status
- Critical maintenance counter

In addition, live temperature and vibration graphs are displayed using Matplotlib. These graphs are updated automatically whenever new telemetry data is received from the MQTT broker. **WARNING** and **CRITICAL** threshold lines are also displayed to provide visual indications of abnormal operating conditions.

To improve operator awareness, the dashboard includes a colour-coded event log that records MQTT messages, maintenance events, command transmissions, and system status updates.

Table 1: Colour-coded Event Logging

Event Type	Colour
Normal	Green
Warning	Yellow
Critical	Red
Maintenance	Purple
Information	Blue

Popup notifications are also generated whenever critical maintenance conditions are detected, ensuring that important maintenance events are immediately visible to the operator.

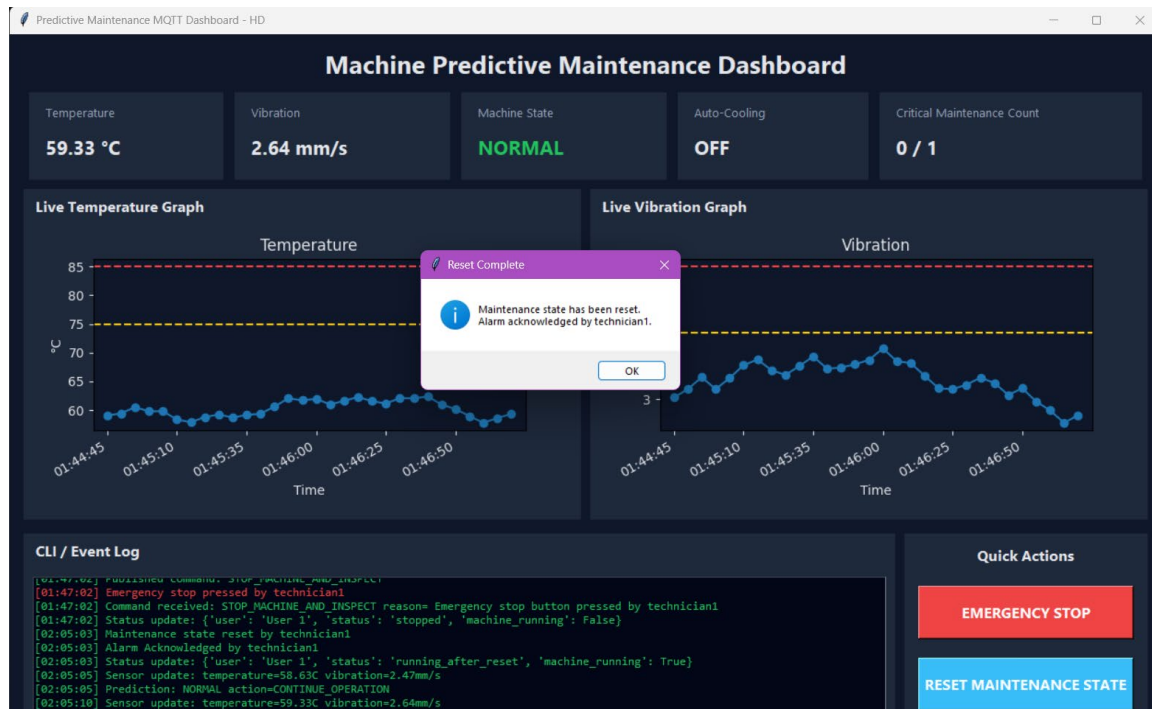


Figure 29: Reset acknowledgement response popped up upon pressing blue button under quick actions

Besides monitoring functions, the dashboard provides several operator control features such as Emergency Stop and Maintenance Reset controls. The dashboard publishes messages to the following MQTT topics:

- public/102781776/machine01/command
- public/102781776/machine01/reset
- public/102781776/machine01/ack

These topics support emergency stop operations, maintenance reset requests, and alarm acknowledgement actions.

The Emergency Stop feature allows operators to immediately place the machine into a stopped state when unsafe operating conditions are detected. Once activated, a stop command is transmitted through the MQTT broker and telemetry generation is suspended.

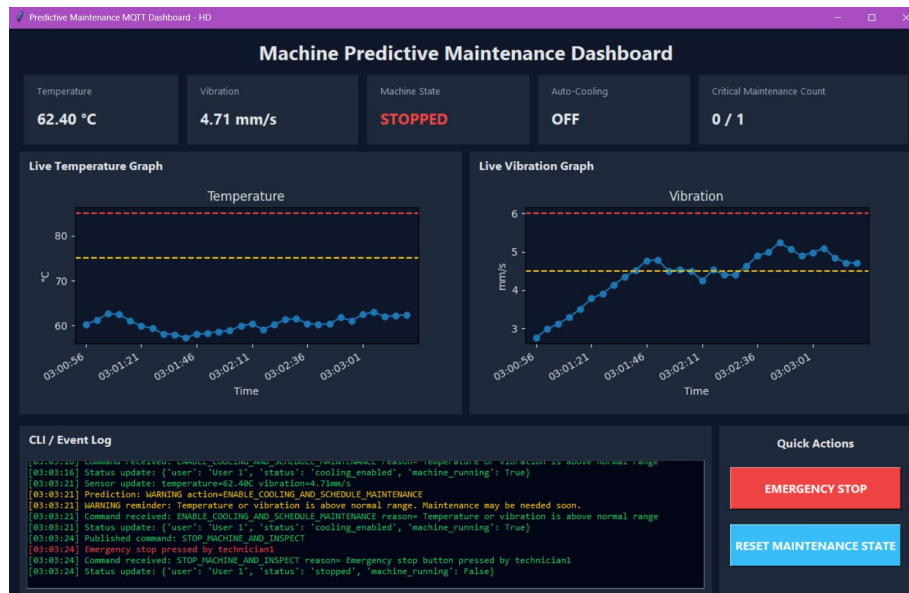


Figure 30: Stop command issued and pressed with red button

Similarly, the Maintenance Reset feature allows authorised personnel to clear maintenance lockout conditions, resets the maintenance counter, acknowledges active alarms, and restores normal machine operation after maintenance activities have been completed.

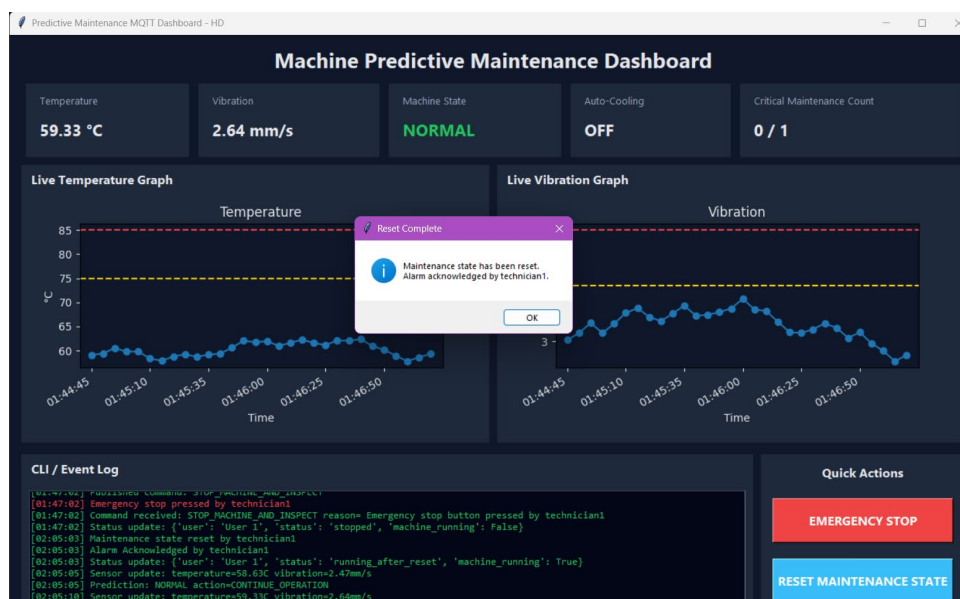


Figure 31: Alarm acknowledged via ack public topic in gui.py after maintenance reset

In addition to telemetry visualisation, several predictive maintenance support functions were implemented within the dashboard. When warning thresholds are exceeded, the dashboard automatically displays an active auto-cooling status to indicate that cooling measures have been recommended and performed immediately.

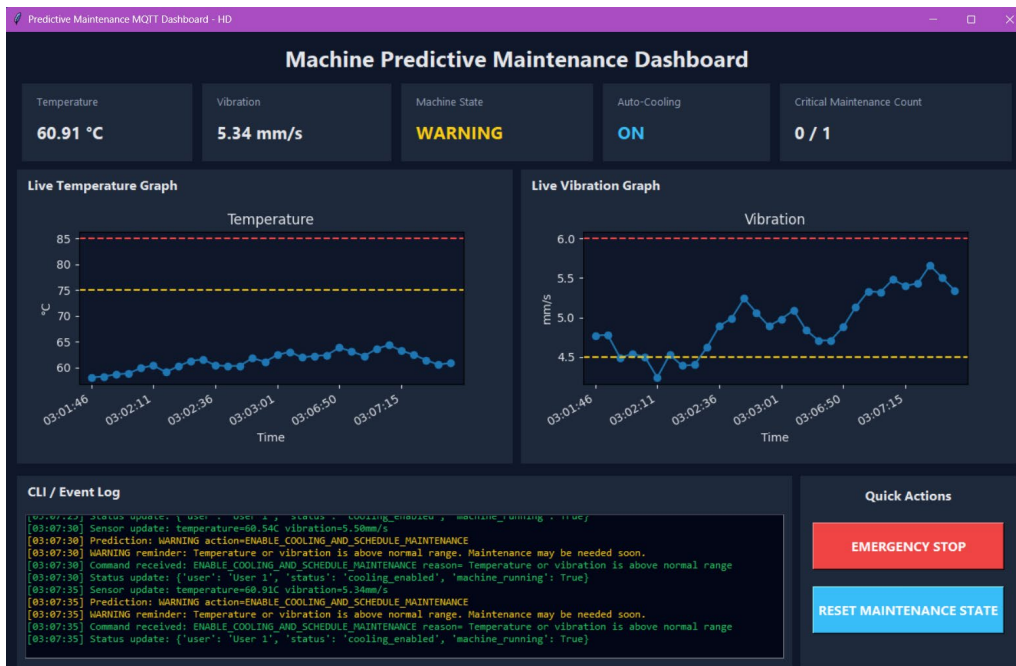


Figure 32: Auto-Cooling ON after entering WARNING machine state

When critical operating conditions are detected, the dashboard increments a critical maintenance counter and records the event within the colour-coded event log. Once the critical maintenance threshold is reached, the machine enters a MAINTENANCE REQUIRED state, a popup alert is displayed to notify the operator, and machine operation is suspended until a maintenance reset is performed.

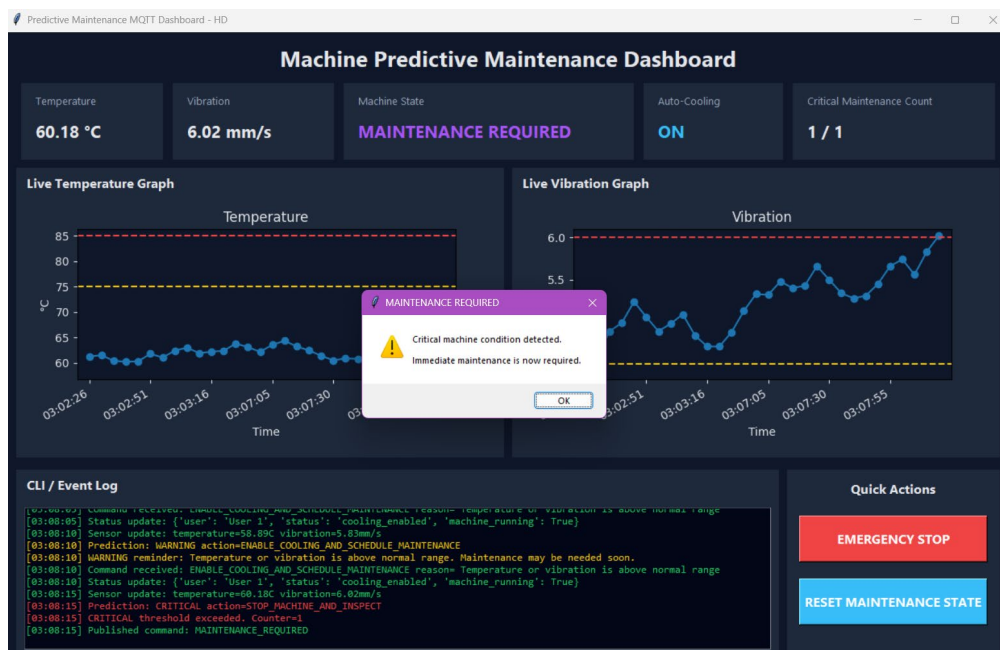


Figure 33: MAINTENANCE_REQUIRED machine state when the machine has reached critical maintenance limit

Upon pressing 'OK', the machine will immediately transition into STOPPED machine state to indicate the machine operation is put into a halt until the maintenance personnel comes to intervene and decide further quick actions according to their preference.

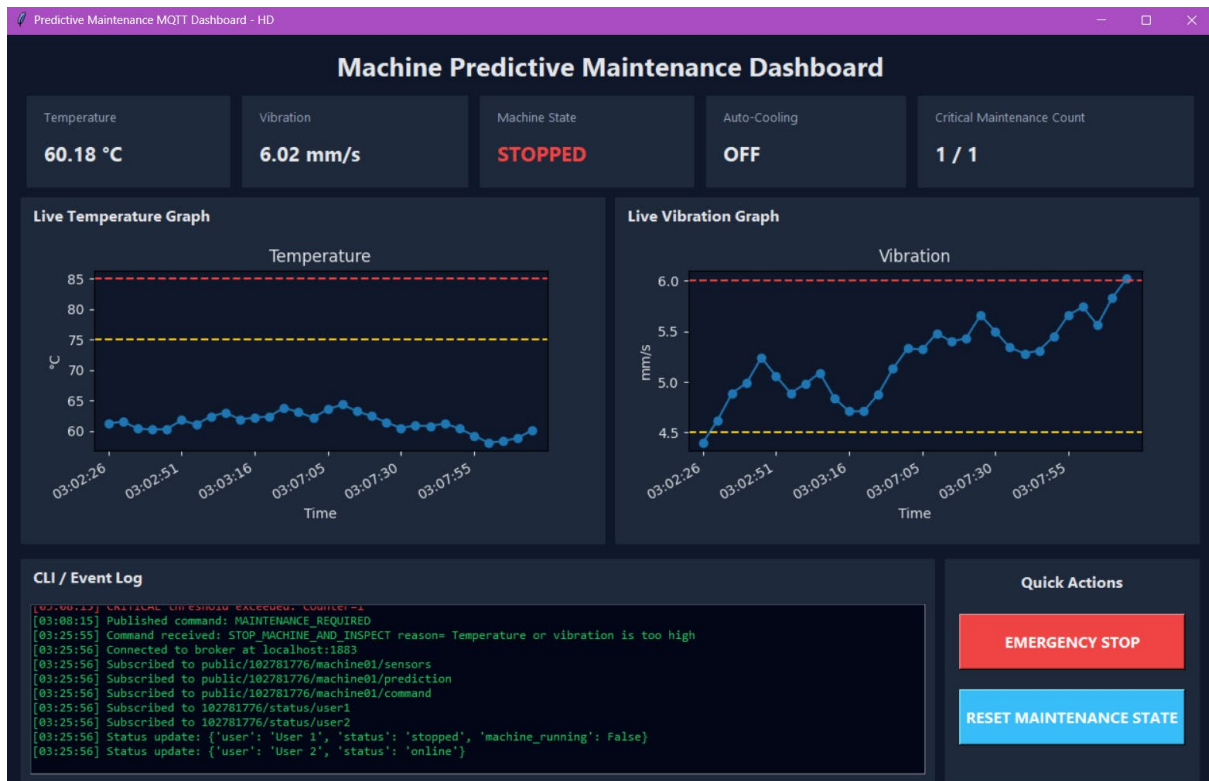


Figure 34: Expected output after clicking OK on the popup alert

GUI Dashboard Client provides the visualisation and operator interaction layer of the predictive maintenance architecture. By combining live telemetry monitoring, graphical visualisation, maintenance alerts, operator controls, and MQTT communication into a single interface, the dashboard enhances usability and supports effective machine condition monitoring and maintenance decision-making.

3.0 Running the Program

The predictive maintenance system can be executed using either a local MQTT broker running on the same computer or a network-accessible MQTT broker connected through a local Wi-Fi network.

Before running the system, ensure that Mosquitto MQTT Broker, Python, and all required Python libraries have been installed successfully. The same applies to establishing a test connection with username and password having the same student ID. For example, username and password must be the same. Consider typing a few messages to the local broker to see the messages received.

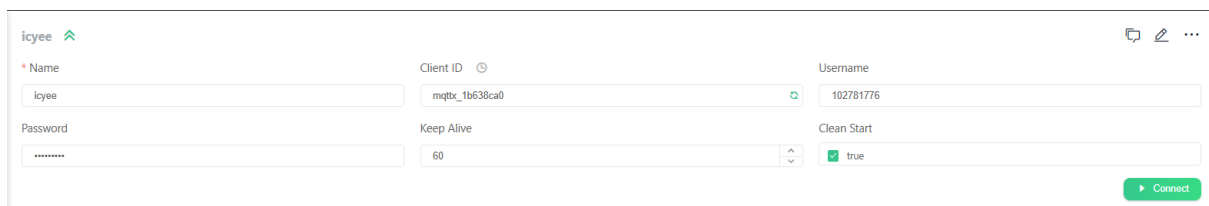
The image shows the MQTTX configuration window. It has a title bar with 'icylee' and a green icon. Below the title bar, there are several input fields: 'Name' with 'icylee', 'Client ID' with 'mqttx_1b638ca0', 'Username' with '102781776', 'Password' with a masked password '*****', 'Keep Alive' with '60', and 'Clean Start' with a checked checkbox and the value 'true'. There is a green 'Connect' button at the bottom right.

Figure 35: MQTTX Configuration to show MQTT connectivity and ability to process information across topics

The following steps to run the system are as follows:

Step 1: Configure the MQTT Broker Address

Open the following files:

- publisherHD.py
- subscriberHD.py
- gui_dashboardHD.py

Locate the broker configuration across each file:

```
broker = "localhost" # use 192.168.12.100 as local broker IP
```

Figure 36: Current setup for broker IP

Option A: Localhost Deployment

Use the following configuration if the MQTT broker and all MQTT clients are running on the same computer:

```
broker = "localhost"
```

Option B: Wi-Fi / Local Network Deployment

Use the following configuration if the MQTT broker is hosted on another device within the same network:

```
broker = "192.168.12.100"
```

Replace the IP address with the actual MQTT broker IP address if necessary.

For Wi-Fi deployment, ensure that:

- All devices are connected to the same network.
- The MQTT broker machine allows incoming connections on TCP port 1883.
- Windows Firewall or antivirus software does not block MQTT communication.

Step 2: Run subscriberHD then publisherHD

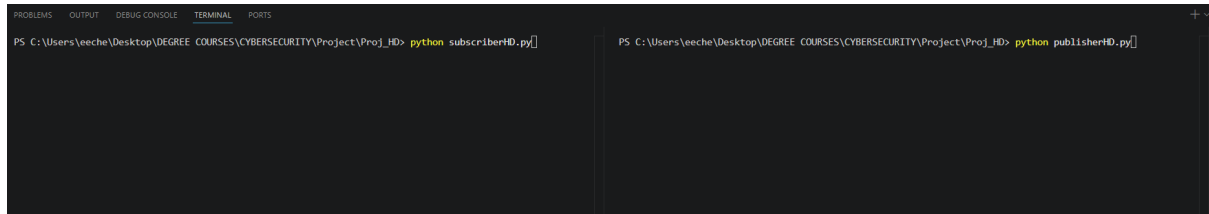


Figure 37: Expected configuration for two users running on different terminals

In VS Code or Command Prompt

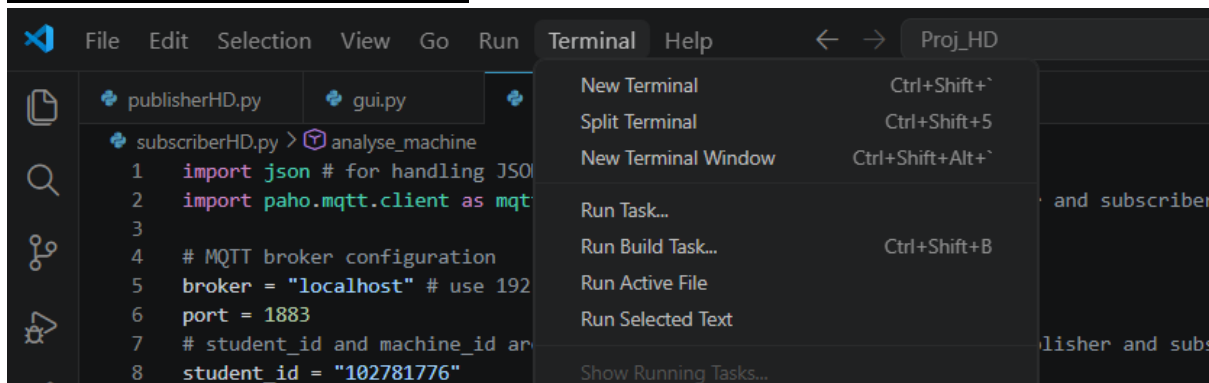


Figure 38: Terminal tab in VS Code

Create a new terminal window first and split the terminals (Terminal Tab > Split Terminal / Ctrl+Shift+5), then execute this:

```
python subscriberHD.py
```

After that, execute:

```
python publisherHD.py
```

This will now run the sensor readings in the terminal window as shown in Figure 37 below.

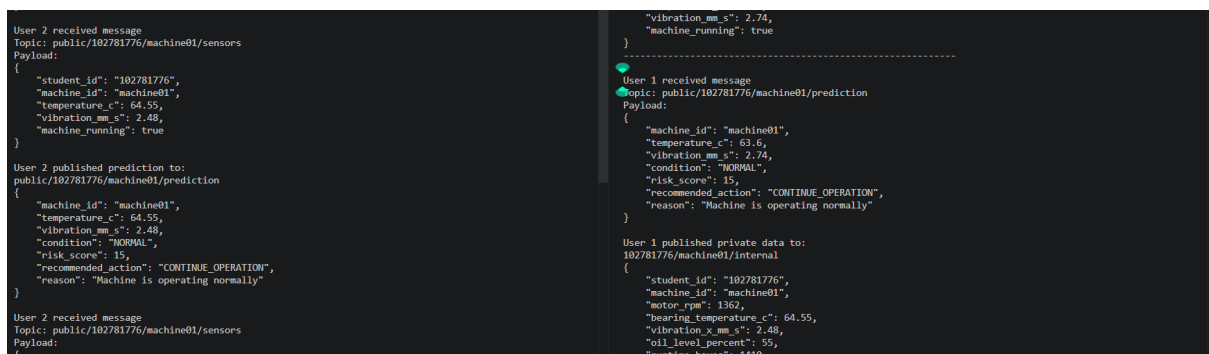


Figure 39: Expected terminal output before running GUI

Step 4: Run the GUI Dashboard Client

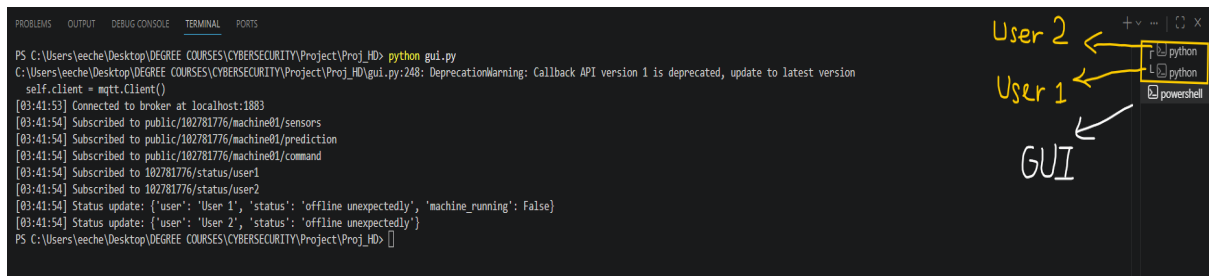


Figure 40: GUI terminal

Create another terminal window and execute:

```
python gui_dashboardHD.py
```

The dashboard will automatically connect to the MQTT broker and display:

- Live temperature readings
- Live vibration readings
- Machine operating state
- Auto-cooling status
- Critical maintenance counter
- Event logs
- Real-time telemetry graphs

Step 5: Monitor MQTT Communication Using MQTTX

Open MQTTX and create a new MQTT connection as shown in Figure 33.

Broker address:

localhost or 192.168.12.100

Port: 1883

Subscribe to the following topics outlined below:

102781776/#

public/102781776/#

These subscriptions allow MQTTX to display all MQTT messages generated by the system publicly and privately. The MQTT responses can also be opted to separate into different subtopics for better visualization as shown in Figure 40.

102781776/mach... QoS 0	102781776/machine01/internal
public/10278177... QoS 0	Public/102781776/machine01/sensors
public/10278177... QoS 0	Public/102781776/machine01/prediction
public/10278177... QoS 0	Public/102781776/machine01/command
102781776/statu... QoS 0	102781776/status/user1
102781776/statu... QoS 0	102781776/status/user2
public/10278177... QoS 0	Public/102781776/machine01/reset

Figure 41: Subscribed topics in MQTTX

Step 6: Verify System Operation

Once all clients are running successfully:

1. Machine telemetry data will be generated by User 1.
2. Predictive maintenance analysis will be performed by User 2.
3. Machine conditions will be displayed on the GUI dashboard.
4. Warning and critical events will be logged automatically.
5. Emergency stop and maintenance reset functions can be tested through the dashboard interface.
6. MQTT messages can be observed through MQTTX subscriptions.

The system is considered operational when telemetry data, predictive maintenance responses, and dashboard updates are continuously exchanged through the MQTT broker.